
CAHIER DES CHARGES TECHNIQUE

Plateforme De Gestion Des Goodies Digicheese

Le 17/04/2026

Thibault ODOR

Victor ODIN

Julien MILHAU VILLAR

1. Gestion documentaire	3
Introduction et contexte du projet	3
Identification du projet	3
Versioning et historique des modifications	3
Circuit de validation	4
Glossaire	4
2. Architecture technique	6
Schéma global de l'architecture	6
Infrastructure	7
Serveur d'application (Back-end)	7
Serveur de base de données	8
Serveur front-end	8
Pipeline CI/CD	8
Outils de supervision	8
3. Bases de données	9
Modèle physique des données (MPD)	9
Tables principales du module stocks	9
Colonnes clés de la table stock_ligne	10
Volumétrie estimée	10
Exemples de données représentatives	11
Table objet (extrait)	11
Table stock_ligne (extrait)	11
Conformité RGPD	11
4. Flux et interactions	12
Cartographie des modules	12
Spécifications API	12
Principes généraux	12
Endpoints d'authentification	12
Endpoints Administration (rôle : admin)	13
Endpoints Opérateur Colis (rôle : colis)	13
Formats d'échange	13
Format de requête	13
Format de réponse – exemples	14
Spécifications UI	14
Interface utilisateur web	14
Ergonomie et responsive	14
Accessibilité	14
5. Spécifications de l'interface utilisateur	15
Ergonomie générale	15
Pages et composants du module gestion des stocks	15
Responsive Design	16
Accessibilité	16
6. Contraintes et sécurité	17
Compatibilité navigateurs	17

Authentification et gestion des accès	17
Mécanisme d'authentification	17
Gestion de session	18
Gestion des mots de passe	19
Gestion des autorisations	19
Contraintes de sécurité complémentaires	20
Sécurité applicative	21
7. Plan de développement	22
DoD / DoR	23
Definition of Ready (DoR)	23
Definition of Done (DoD)	24
Organisation du développement	25
8. Diagrammes et annexes	26
Diagrammes UML	26
Diagramme de contexte général	26
Diagramme de cas d'utilisation (Use Case)	27
Diagramme de classes	28
Diagramme de séquence	29
Diagramme entité-relation (ERD / MPD)	30
Diagramme BPMN	31
Schéma réseau	32

1. Gestion documentaire

Introduction et contexte du projet

La Fromagerie Digicheese est une TPE spécialisée dans la vente de produits fromagers et la fidélisation de sa clientèle via un système de cadeaux. Pendant plus de vingt ans, la gestion de ce programme a reposé sur une application développée sous Microsoft Access 2000 en VBA un outil devenu obsolète, instable, non maintenu et incompatible avec les systèmes d'exploitation récents.

Face à ces limitations, la fromagerie a lancé en 2022 un projet de refonte complète de son système d'information. L'objectif est de remplacer cette application vieillissante par une API backend moderne, accessible via une interface web interne, et capable de gérer l'ensemble du cycle de vie des commandes de cadeaux fidélité : de la saisie des clients à l'expédition des colis, en passant par la gestion des stocks et l'administration des paramètres métier.

L'application est structurée autour de trois profils utilisateurs aux périmètres bien distincts : l'administrateur, qui gère les référentiels métier et les comptes utilisateurs ; l'opérateur colis, responsable de la gestion des clients, commandes et expéditions ; et l'opérateur stock, chargé du suivi des inventaires. Ces rôles sont cumulables et contrôlés par un système de permissions intégré à l'application.

Identification du projet

Nom du projet	DigiCheese Refonte Goodies
Référence	CDCT-DigiCheese-v1.0
Date de création	Avril 2026
Équipe projet	Julien Milhau Villar, Thibault Odor, Victor Odin
Commanditaire	Fromagerie DIGICHEES
Référent DSI	Christophe Germain – cgermain@diginamic.fr
Technologies	Python 3.11 / Flask / Flask-SQLAlchemy / Flask-Login / MySQL / Flasgger (Swagger)
Dépôt	Projet TP7 – Diginamic Lead Dev/DevOps 2025-2026

Versioning et historique des modifications

Version	Date	Description
V 0.1	12/10/2022	Brouillon initial
V 1.0	17/10/2022	Première version finalisée du cahier des charges

Version	Date	Description
V 1.2	18/10/2022	Version finale – corrections mises en page
V 2.0	19/10/2022	Version avant-vente client
V 3.0	Avril 2026	Mise à jour CDCT – Sprint 1 backend livré (API REST Flask, authentification, CRUD admin & colis, gestion stock)

Circuit de validation

Version	Nom / Qualité	Date	Commentaires
V 1.0	C. Germain – Chef de projet	17/10/2022	Validation contenu + ajustements mise en page
V 2.0	Équipe DigiCheese	Avril 2026	Validation Sprint 1 – backend API opérationnel, tests pytest passants

Glossaire

API REST

Interface de programmation applicative respectant les contraintes REST (Representational State Transfer). C'est le point d'entrée HTTP principal du backend DigiCheese, via lequel toutes les opérations métier sont exposées.

Flask

Micro-framework web Python utilisé pour construire l'API backend. Flask gère le routage HTTP, les sessions utilisateur et l'injection des extensions (SQLAlchemy, Login, Swagger).

SQLAlchemy

ORM (Object Relational Mapper) Python permettant d'interagir avec la base de données MySQL sans écrire de SQL brut. Chaque table de la base est représentée par un modèle Python héritant de `db.Model`.

Blueprint

Mécanisme Flask permettant de regrouper les routes par domaine fonctionnel. Le projet utilise un Blueprint par module : `auth`, `admin_users`, `colis_commandes`, etc. Chaque Blueprint est enregistré dans la factory `create_app()`.

CRUD

Acronyme désignant les quatre opérations de base sur les ressources : Create (POST), Read (GET), Update (PUT), Delete (DELETE). La quasi-totalité des ressources de l'application expose ces quatre opérations.

Session / Flask-Login

Mécanisme de gestion de la connexion utilisateur. Flask-Login maintient l'état de session via un cookie signé côté serveur. Le decorator `@login_required` protège les routes qui nécessitent une authentification.

Rôle

Niveau de permission associé à un utilisateur, défini dans l'enum `RoleEnum`. Les trois rôles disponibles sont **admin**, **colis** et **stock**. Un même utilisateur peut en cumuler plusieurs via la table de liaison `roles_utilisateur`.

Admin

Rôle réservé à l'administration de l'application. Un utilisateur admin peut gérer les comptes utilisateurs, les communes, les objets/goodies et les conditionnements.

OP-colis (Opérateur colis)

Rôle métier dédié à la gestion des expéditions. L'opérateur colis créé et suit les clients, les commandes et les adresses, génère les mailings et consulte les statistiques d'activité.

OP-stock (Opérateur stock)

Rôle métier dédié à la gestion des inventaires. L'opérateur stock saisit les mouvements de stock, gère les lignes de stock par objet et peut déclencher une mise à jour annuelle des quantités.

Flasgger / Swagger

Flasgger est l'extension Flask qui génère automatiquement une interface Swagger UI à partir des docstrings des routes. Elle est accessible à l'URL `/apidocs` et permet de tester tous les endpoints sans client externe.

Objet / Goodie

Article cadeau de fidélité distribué aux clients de la fromagerie (t-shirt, casquette, etc.). Chaque objet possède un libellé, une taille, un poids et un indicateur de disponibilité (`bl_indispo`).

Commande

Commande passée par un client, identifiée par une date, un nombre de colis et un conditionnement. Elle est composée de plusieurs lignes de détail (`DetailCommande`), chacune associant un objet à une quantité.

Conditionnement

Type d'emballage utilisé pour expédier une commande (carton, palette...). Chaque conditionnement a un libellé, un poids et un ordre d'impression utilisé pour générer les bons de préparation.

Stock ligne

Entrée dans le stock associant un objet à un emplacement de stockage. Elle comporte un libellé, une quantité disponible et des dates de début/fin de validité permettant d'historiser les mouvements.

pytest

Framework de tests unitaires et d'intégration Python. Le projet utilise pytest avec pytest-flask et une base SQLite en mémoire pour valider les routes, les contrôles d'accès par rôle et la logique d'authentification, sans dépendre de la base MySQL de production.

2. Architecture technique

Description global de l'architecture

L'architecture retenue est une architecture en couches de type client-serveur à trois niveaux (3-tiers). Elle se décompose comme suit :

- Couche de présentation (Front-end) : application web monopage (SPA) développée en Vue.js, accessible depuis un navigateur web moderne. Cette couche gère l'affichage des données, la navigation et les interactions utilisateur.
- Couche métier (Back-end) : API REST développée en Python avec le micro framework Flask, déjà partiellement implémenté lors du bloc précédent. Elle expose les endpoints nécessaires à la gestion des stocks, des commandes et des utilisateurs.
- Couche de persistance (Base de données) : base de données relationnelle PostgreSQL hébergée sur un serveur dédié. Elle stocke l'ensemble des données métier de la Fromagerie DigiCheese.

Le front-end communique exclusivement avec le back-end via des appels HTTP/HTTPS à l'API REST. Aucun accès direct à la base de données n'est autorisé depuis le navigateur. L'ensemble des communications est chiffré en TLS 1.2 minimum.

Environnements

Quatre environnements distincts sont définis pour couvrir l'ensemble du cycle de vie de l'application. Chaque environnement est isolé des autres afin d'éviter toute contamination de données ou de configuration.

Environnement	Objectif	Accès	Base de données
DEV (Développement)	Développement itératif des fonctionnalités. Utilisé au quotidien par les développeurs.	Équipe de développement uniquement	Base locale ou partagée avec jeu de données de test
PP (Preprod)	Validation intégration front-end / back-end, tests fonctionnels et unitaires automatisés. Recettage et validation User Storie.	Équipe de développement Testeurs Client (Fromagerie DigiCheese)	Base dédiée, données proches de la production sur base de fixtures anonymisées
PROD (Production)	Environnement cible final. Données réelles de la fromagerie.	Utilisateurs finaux autorisés	Base de production sécurisée et sauvegardée

Infrastructure

L'infrastructure est hébergée sur des serveurs virtuels. Les choix retenus privilégient la simplicité de déploiement, la maintenabilité et la maîtrise des coûts, adaptée à la taille d'une TPE comme la Fromagerie DigiCheese.

Serveur d'application (Back-end)

Paramètre	Valeur
Type	Machine virtuelle Linux (Ubuntu 24.04 LTS)
vCPU	2 cœurs
Mémoire RAM	4 Go
Stockage système	50 Go SSD
Middleware	Python 3.14, Flask 3.1.X
Port d'écoute	8080 (interne) — 443 (exposition HTTPS via reverse proxy)
Reverse proxy	Traefik — gestion du SSL/TLS et redirection des requêtes
Certificat TLS	Let's Encrypt (renouvelé automatiquement via Certbot)
Serveur d'application	Gunicorn 25.X

Serveur de base de données

Paramètre	Valeur
Type	Machine virtuelle Linux (Ubuntu 24.04 LTS)
vCPU	2 cœurs
Mémoire RAM	4 Go
Stockage	30 Go SSD (système) + 100 Go SSD (données)
SGBD	PostgreSQL 18
Port d'écoute	5432 (accès restreint au réseau interne uniquement)
Sauvegarde	Dumps journaliers automatisés (pg_dump), rétention 30 jours

Serveur front-end

Le front-end étant une SPA Vue.js, le build de production consiste en un ensemble de fichiers statiques (HTML, CSS, JS). Ces fichiers sont servis directement par Unicorn depuis le même serveur d'application, dans un répertoire dédié.

Pipeline CI/CD

Un pipeline d'intégration et de déploiement continu est mis en place afin d'automatiser les étapes de build, de test et de déploiement. L'outil retenu est GitHub Actions, intégré directement au dépôt Git du client.

- Déclencheur: tout push sur la branche develop (environnement PREPROD) ou master (environnement PROD).
- Étapes du pipeline : (1) Installation des dépendances Python puis des dépendances npm, (2) Exécution des tests unitaires et d'intégration, (3) Build de production Vue (npm run build), (4) Analyse de qualité du code (ESLint + Prettier), (5) Déploiement automatique vers l'environnement cible via SSH.
- Notification: en cas d'échec du pipeline, une notification est envoyée par email aux membres de l'équipe.

Outils de supervision

Afin d'assurer la disponibilité et la performance de l'application en production, les outils de supervision suivants sont préconisés :

Outil	Usage	Justification
Grafana + Prometheus	Tableaux de bord de métriques (CPU, RAM, temps de réponse API, taux d'erreur)	Solution open source, très répandue
Unicorn accessLogs	Suivi des requêtes HTTP (statuts, volumes, IP)	Natif Unicorn, pas de surcoût
pg_stat_activity	Supervision des requêtes PostgreSQL actives	Outil natif PostgreSQL
Sentry	Remontée automatique des erreurs JavaScript côté front-end	Facilite le débogage en production

3. Bases de données

Modèle physique des données (MPD)

La base de données PostgreSQL de la Fromagerie DigiCheese hérite de la modélisation réalisée lors du bloc précédent (Modélisation). Le présent CDCT se concentre sur les tables directement impliquées dans le module de gestion des stocks. Le schéma complet du MPD est disponible en annexe.

Tables principales du module stocks

Table	Description	Clé primaire	Relations principales
utilisateur	Comptes des utilisateurs de l'application (toutes les connexions)	idUtil (INT, PK, AUTO)	Lié à rôle (n-n via accomplir)
rôle	Rôles fonctionnels : Admin, OP-colis, OP-stocks	idRole (INT, PK, AUTO)	Lié à utilisateur (n-n)
objet	Catalogue des articles/cadeaux de fidélité gérés en stock	idObjet (INT, PK, AUTO)	Lié à stock_ligne, conditionner, boutique
stock	En-tête de stock (exercice annuel)	idStock (INT, PK, AUTO)	Lié à stock_ligne (1-n)
stock_ligne	Lignes de stock détaillant la quantité par objet et par période	idStockLigne (INT, PK, AUTO)	Lié à objet (n-1), stock (n-1)
boutique	Représentation de la boutique physique et de ses niveaux de stock	idBoutique (INT, PK, AUTO)	Lié à objet (n-n via appartenir)
mise_a_jour	Historique des mises à jour annuelles du stock	idMaj (INT, PK, AUTO)	Lié à stock_ligne (1-n)
commande	Commandes passées par les clients via points de fidélité	idCommande (INT, PK, AUTO)	Lié à client, detail_commande
detail_commande	Lignes de commande (quantité, conditionnement)	Clé composite (idCommande, idCond)	Lié à commande, conditionnement
conditionnement	Définition des emballages (poids, libellé, ordre)	idCond (INT, PK, AUTO)	Lié à objet (n-n)

Colonnes clés de la table stock_ligne

Colonne	Type SQL	Contrainte	Description
idStockLigne	INTEGER	PK, NOT NULL, AUTO INCREMENT	Identifiant unique de la ligne de stock
idObjet	INTEGER	FK → objet.idObjet, NOT NULL	Référence à l'article concerné
idStock	INTEGER	FK → stock.idStock, NOT NULL	Référence à l'exercice de stock
libelle	VARCHAR(255)	NOT NULL	Libellé descriptif de la ligne
dateDebut	DATE	NOT NULL	Date de début de validité de la ligne
dateFin	DATE	NULL autorisé	Date de fin (NULL si stock en cours)
quantiteStock	INTEGER	NOT NULL, DEFAULT 0, CHECK >= 0	Quantité disponible en stock

Volumétrie estimée

La fromagerie DigiCheese est une TPE. Les volumes de données attendus sont modestes. Les estimations suivantes sont fournies à titre indicatif pour dimensionner correctement l'infrastructure.

Table	Nombre d'enregistrements estimé (an 1)	Croissance annuelle estimée	Justification
utilisateur	<20	Stable (faible rotation du personnel)	TPE avec quelques salariés
client	500 — 2 000	+ 200/an	Base clients fidélité, croissance progressive
objet	50 — 200	+ 20/an	Catalogue de cadeaux relativement stable
stock_ligne	500 — 2 000	x2/an	Une ligne par objet par mois ou période
commande	1000 — 5 000	x1.5/an	Commandes cadeaux fidélité des clients
detail_commande	3000 — 15 000	x1.5/an	Plusieurs lignes par commande en moyenne

La base de données ne dépassera pas quelques dizaines de mégaoctets dans les 3 premières années d'exploitation. Le dimensionnement retenu (100 Go de stockage secondaire) offre une très large marge de sécurité.

Exemples de données représentatives

Les jeux de données suivants sont fournis à titre d'exemple pour les phases de développement et de test. Ils reflètent des cas d'usage réels de la fromagerie.

Table objet (extrait)

idObjet	libelle	poids	blindispo
1	Plateau De Fromage (Bois)	0.45	false
2	Pins fromagerie Digicheese	0.80	false
3	T-shirt fromagerie Digicheese	1.20	false

Table stock_ligne (extrait)

idStockLigne	idObjet	idStock	libelle	dateDebut	dateFin	quantiteStock
1	1	1	Plateau Pour Fromage (Bois)	01/01/2026	null	148
2	2	1	Pins fromagerie Digicheese	01/01/2026	null	73
3	3	1	T-shirt fromagerie Digicheese	01/01/2026	null	210

Conformité RGPD

La Fromagerie DigiCheese collecte et traite des données à caractère personnel de ses clients (nom, prénom, adresse, email). Ces traitements sont soumis au Règlement Général sur la Protection des Données (RGPD — Règlement UE 2016/679). Les mesures suivantes sont mises en œuvre :

Mesure RGPD	Implémentation technique
Minimisation des données	Seules les données strictement nécessaires à la gestion des commandes et du programme de fidélité sont collectées. Aucun champ superflu.
Droit d'accès et de rectification	Interface administrateur permettant la consultation et la modification des données client. API dédiée exposant ces fonctionnalités.
Droit à l'effacement	Procédure d'anonymisation des données client sur demande (remplacement des données personnelles par des valeurs neutres, conservation de l'historique des commandes de manière anonymisée).
Sécurité des données	Chiffrement TLS en transit, hachage des informations sensibles (bcrypt, facteur de coût 12), accès base de données restreint au réseau interne.
Durée de conservation	Données clients conservées 3 ans après le dernier achat, puis anonymisées automatiquement via un job planifié.
Journalisation des accès	Log des accès aux données sensibles (consultation de fiches clients, exports) avec horodatage et identifiant utilisateur.
Consentement	Mention d'information RGPD présentée lors de la création d'un compte client. Consentement explicite requis pour les communications marketing.

4. Flux et interactions

Cartographie des modules

L'application DigiCheese est structurée autour de trois domaines fonctionnels correspondant aux rôles utilisateurs, chacun exposé via des Blueprints Flask distincts. La couche d'accès aux données est mutualisée via des repositories héritées de **BaseRepository**.

Catégorie	Module	Blueprint Flask	URL préfixe	Rôle requis
Authentification	Authentification	auth	/login, /api-login, /logout	Aucun (public)
Administration	Utilisateurs	admin_users	/admin/users	admin
Administration	Objets	admin_objets	/admin/objets	admin
Administration	Communes	admin_communes	/admin/communes	admin
Administration	Conditionnements	admin_conditionnements	/admin/conditionnements	admin
Colis	Clients	colis_clients	/colis/clients	colis
Colis	Adresses	colis_adresses	/colis/adresses	colis
Colis	Commandes	colis_commandes	/colis/commandes	colis
Colis	Détails commande	colis_detail_commandes	/colis/detail_commandes	colis
Colis	Mailing	colis_mailler	/colis/mailler	colis
Colis	Statistiques	colis_statistique	/colis/statistique	colis
Documentation API	Flasgger (Swagger)	Flasgger	/apidocs	Aucun (interne)

Spécifications API

Principes généraux

- Toutes les routes métier utilisent le format JSON (Content-Type: application/json)
- L'authentification utilise les sessions Flask-Login (cookie de session côté serveur)
- Les erreurs sont retournées avec le code HTTP approprié et un objet JSON { "error": "message" }
- La documentation interactive est disponible via Swagger UI à l'URL /apidocs

Endpoints d'authentification

Méthode	Route	Auth requise	Description
POST	/api-login	Non	Connexion via JSON { email, password } – retourne l'objet utilisateur avec ses rôles
GET	/api-logout	Oui	Déconnexion – invalide la session courante
GET/POST	/login	Non	Page HTML de connexion (interface web Bulma CSS)
GET/POST	/signup	Non	Page HTML d'inscription (création compte utilisateur)

Endpoints Administration (rôle : admin)

Méthode	Route	Description
GET	/admin/users/	Liste tous les utilisateurs
GET	/admin/users/	Détail d'un utilisateur par ID
POST	/admin/users/add	Crée un utilisateur { email, name, password }
PUT	/admin/users/update/	Modifie un utilisateur (password hashé automatiquement)
DELETE	/admin/users/delete/	Supprime un utilisateur
CRUD	/admin/objets/*	CRUD complet sur les objets/goodies (libellé, taille, poids, disponibilité)
CRUD	/admin/communes/*	CRUD sur les communes (CP, nom, département)
CRUD	/admin/conditionnements/*	CRUD sur les conditionnements (libellé, poids, ordre d'impression)

Endpoints Opérateur Colis (rôle : colis)

Méthode	Route	Description
CRUD	/colis/clients/*	CRUD clients (email, adresse_id)
CRUD	/colis/adresses/*	CRUD adresses (3 lignes + commune_cp)
CRUD	/colis/commandes/*	CRUD commandes (date, timbre_client, nb_colis, client_id, conditionnement_id)
CRUD	/colis/detail_commandes/*	CRUD détails de commande (quantité, colis, commentaire, objet_id, commande_id)
GET	/colis/mailler/template	Retourne un template de mail client personnalisable (sujet + corps Jinja2)
GET	/colis/statistique/statistique	Retourne les statistiques d'activité (total colis, livrés, en attente, taux de livraison)

Formats d'échange

Format de requête

- Toutes les requêtes POST et PUT envoient du JSON dans le corps de la requête
- L'en-tête Content-Type: application/json est obligatoire pour les appels API
- Les champs manquants ou invalides déclenchent une réponse 400

Format de réponse – exemples

Réponse succès – objet utilisateur :

```
{ "id": 1, "name": "admin", "email": "admin@gmail.com", "roles": ["admin"] }
```

Réponse erreur :

```
{ "error": "Utilisateur non trouvé" }
```

Code HTTP	Signification
200	Succès – ressource retournée ou opération réussie
201	Création réussie – l'ID de la nouvelle ressource est retourné
400	Données invalides ou champs manquants dans la requête
401	Non authentifié – l'utilisateur n'est pas connecté
403	Accès refusé – rôle insuffisant
404	Ressource introuvable en base de données

Spécifications UI

Interface utilisateur web

Le projet expose un ensemble de pages HTML minimalistes destinées à la démonstration et aux tests manuels. Ces pages sont servies directement par Flask.

Page	Route	Description
Accueil	/	Page d'accueil générale
Login	/login	Formulaire de connexion HTML (email + mot de passe)
Sign Up	/signup	Formulaire d'inscription
Profil	/profile	Page personnelle – accès réservé aux utilisateurs connectés
Swagger UI	/apidocs	Interface interactive de test et documentation de tous les endpoints API

Ergonomie et responsive

- Navigation dynamique (Login/Signup ou Profile/Logout selon connexion)
- Messages flash via notifications
- Interface principale opérateur : API REST via Swagger, Postman ou frontend

Accessibilité

- Champs HTML avec placeholder
- Documentation API automatique via Swagger/Flasgger
- Messages d'erreur explicites en français

5. Spécifications de l'interface utilisateur

Ergonomie générale

L'interface utilisateur doit être simple, claire et efficace. La cible principale est des opérateurs non-techniciens (salariés de la fromagerie). Les principes ergonomiques suivants guident la conception :

- Principe de moindre surprise : les actions courantes (consulter le stock, modifier une quantité) doivent être accessibles en 2 clics maximum depuis n'importe quelle page de l'application.
- Hiérarchie visuelle claire : les informations critiques (quantités en rupture, alertes de stock faible) sont mises en évidence par une couleur distincte (rouge ou orange) et positionnées en haut des listes.
- Navigation cohérente : une barre de navigation latérale fixe présente les sections de l'application selon le rôle de l'utilisateur connecté. Elle ne disparaît pas lors du défilement.
- Feedback utilisateur : toute action (sauvegarde, mise à jour, suppression) donne lieu à un retour visuel immédiat (message toast de confirmation ou d'erreur) dans un délai maximal de 300 ms.
- Formulaires courts et guidés : les formulaires de saisie sont découpés en étapes logiques. Les champs obligatoires sont clairement signalés. Les messages d'erreur de validation sont affichés inline, directement sous le champ concerné.

Pages et composants du module gestion des stocks

Page / Composant	Rôle fonctionnel	Données affichées	Actions disponibles
Tableau de bord stocks	Vue d'ensemble de l'état du stock courant	Liste des articles avec quantités, indicateurs de rupture, graphique de répartition	Filtrer, trier, accéder au détail
Liste des lignes de stock	Consultation détaillée des lignes pour un exercice donné	Tableau paginé (20 lignes/page), filtres par article et par période	Ajouter, modifier, supprimer une ligne
Formulaire ajout/édition de ligne	Saisie ou modification d'une ligne de stock	Champs : article (liste déroulante), quantité, dates, libellé	Valider, annuler
Gestion des articles (objets)	Administration du catalogue	Liste des articles avec poids, disponibilité	CRUD complet (Admin uniquement)

Mise à jour annuelle	Déclenchement du renouvellement du stock	Résumé du stock N-1, aperçu du stock N à créer	Confirmer la mise à jour
Page d'impression / export	Génération d'un état imprimable du stock	Tableau complet formaté	Imprimer, exporter CSV

Responsive Design

L'application est conçue avec une approche responsive permettant une utilisation sur différentes tailles d'écran. Les points de rupture (breakpoints) retenus suivent les standards de Tailwind CSS ou Bootstrap :

Breakpoint	Largeur écran	Comportement de l'interface
Mobile (sm)	< 640 px	Navigation latérale remplacée par un menu hamburger. Tableaux remplacés par des cartes empilées. Non prioritaire pour ce module (usage principalement bureau).
Tablette (md)	640 -1024 px	Navigation latérale réduite (icônes seules). Tableaux avec colonnes essentielles uniquement.
Bureau (lg/xl)	> 1024 px	Affichage complet : navigation latérale déployée, tableaux avec toutes les colonnes, graphiques visibles.

L'usage principal de l'application se fait sur poste de bureau (ordinateur fixe ou portable). Le responsive mobile est implémenté mais n'est pas le cas d'usage prioritaire pour le module de gestion des stocks.

Accessibilité

L'application vise la conformité des Web Content Accessibility Guideline. Les mesures d'accessibilité implémentées sont les suivantes :

- Contraste des couleurs
- Navigation au clavier : tous les éléments interactifs (boutons, champs, liens) sont accessibles via la touche Tab. L'ordre de focus est logique et cohérent avec la lecture visuelle.
- Attributs ARIA : les composants complexes (modales, menus déroulants, tableaux) sont annotés avec les attributs ARIA appropriés.
- Textes alternatifs : toutes les images fonctionnelles disposent d'un attribut "alt" descriptif.
- Messages d'erreur accessibles : les erreurs de formulaire sont associées aux champs concernés via aria-describedby et annoncées aux lecteurs d'écran.

6. Contraintes et sécurité

Compatibilité navigateurs

L'application devra fonctionner correctement sur les navigateurs modernes couramment utilisés en environnement professionnel. La cible de compatibilité retenue couvre les versions récentes N et N-1 des navigateurs suivants :

Navigateur	Niveau de support	Notes
Google Chrome	Support complet	Navigateur recommandé et prioritaire pour les tests
Mozilla Firefox	Support complet	Comportement attendu équivalent aux autres navigateurs modernes
Microsoft Edge	Support complet	Basé sur Chromium, prioritaire en environnement Windows
Apple Safari	Support secondaire	À valider si usage sur macOS ou iOS dans le périmètre utilisateur
Internet Explorer	Non supporté	Navigateur obsolète, non compatible avec l'architecture retenue

L'interface front-end s'appuie sur des technologies web modernes, notamment HTML5, CSS3, Flexbox, CSS Grid et JavaScript ES2022+. Le processus de build devra être configuré pour garantir la compatibilité avec les navigateurs cibles définis par la politique de support du projet.

Authentification et gestion des accès

Mécanisme d'authentification

L'authentification de l'application repose sur un mécanisme de **JWT (JSON Web Token)** adapté à une architecture front-end / back-end séparée.

Le flux d'authentification est le suivant : l'utilisateur saisit son adresse email et son mot de passe via le formulaire de connexion. Le front-end transmet ces informations au back-end au travers d'une requête sécurisée en HTTPS. Le back-end vérifie la validité des identifiants en comparant le mot de passe fourni avec l'empreinte stockée en base de données. En cas de succès, il génère un **jeton d'accès JWT** signé avec l'algorithme **HMAC-SHA256 (HS256)** et retourne ce jeton au client.

Le jeton contient les informations minimales nécessaires à l'identification de l'utilisateur et au contrôle des autorisations :

Claim JWT	Valeur	Description
sub	ID utilisateur	Identifiant unique de l'utilisateur
email	Adresse email	Identifiant fonctionnel de connexion
roles	Tableau de rôles	Liste des rôles attribués à l'utilisateur
iat	Timestamp Unix	Date d'émission du jeton
exp	Timestamp Unix	Date d'expiration du jeton

Dans la mesure où les rôles utilisateur sont cumulables, le claim **roles** devra être modélisé sous forme de tableau, par exemple : **["Admin", "OP-colis"]**.

Gestion de session

La gestion de session devra être conçue de manière à limiter les risques de compromission côté client, tout en garantissant une expérience utilisateur fluide.

Le **jeton d'accès** ne devra pas être stocké dans des mécanismes de stockage persistants du navigateur tels que **localStorage** et **sessionStorage**. En effet, ces espaces sont accessibles au code JavaScript exécuté dans la page. En présence d'une faille de type **XSS (Cross-Site Scripting)**, un script malveillant pourrait lire leur contenu et récupérer le jeton d'authentification, permettant ainsi une usurpation de session.

Le jeton d'accès sera donc conservé **uniquement en mémoire applicative** côté front-end, par exemple dans l'état global de l'application. Cette approche réduit sa surface d'exposition, car le jeton n'est pas écrit durablement dans le navigateur et disparaît lors d'un rechargement complet de la page ou de la fermeture de l'onglet.

Afin d'assurer la continuité de session sans conserver un jeton d'accès trop long, le mécanisme recommandé repose sur deux éléments complémentaires :

- un **access token** de courte durée de vie, par exemple **15 à 30 minutes** ;
- un **refresh token** de durée plus longue, stocké dans un **cookie sécurisé**.

Ce cookie devra être configuré avec les attributs suivants :

- **Secure** : le cookie n'est transmis que via une connexion HTTPS ;
- **SameSite** : le cookie est protégé contre certains scénarios d'envoi cross-site, ce qui contribue à limiter les risques liés aux attaques CSRF.

Le fonctionnement attendu est le suivant : lorsque le jeton d'accès expire, le front-end déclenche une demande de renouvellement auprès du back-end. Si le **refresh token** est toujours valide, un nouveau jeton d'accès est émis sans interrompre la session utilisateur. En revanche, si le renouvellement échoue ou si le refresh token a expiré, l'utilisateur est automatiquement redirigé vers la page de connexion.

Cette stratégie permet de concilier **sécurité**, en limitant l'exposition des jetons côté client, et **confort d'utilisation**, en évitant une reconnexion trop fréquente de l'utilisateur.

Gestion des mots de passe

Les mots de passe des utilisateurs ne devront jamais être stockés en clair dans la base de données. Ils ne transiteront qu'au travers d'un canal sécurisé en **HTTPS** et seront traités exclusivement côté back-end. Lors de leur création ou de leur modification, ils seront hachés à l'aide de l'algorithme **bcrypt**, avec un **facteur de coût de 12**, avant d'être enregistrés.

Le système devra également appliquer une politique minimale de robustesse des mots de passe, incluant une **longueur minimale**, un **niveau de complexité suffisant** ainsi que le **refus des mots de passe trop faibles ou trop prévisibles**. Si une fonctionnalité de réinitialisation est prévue, celle-ci devra reposer sur une procédure sécurisée.

L'objectif de ces mesures est de limiter les risques de compromission des comptes utilisateurs et de renforcer la sécurité globale de l'application.

Gestion des autorisations

L'accès aux différentes fonctionnalités de l'application sera contrôlé par un mécanisme de **RBAC (Role-Based Access Control)**. Ce modèle permet d'attribuer à chaque utilisateur un ou plusieurs rôles, lesquels déterminent les actions et les modules auxquels il peut accéder.

Le contrôle des autorisations sera implémenté **côté back-end**, qui constituera la référence de sécurité. Toutes les vérifications d'accès à une ressource ou à une action métier devront être réalisées côté serveur avant exécution du traitement. Le front-end pourra également effectuer un contrôle de rôle afin de masquer les menus, boutons ou actions non autorisés pour l'utilisateur connecté. Cette vérification côté client ne constituera toutefois qu'une mesure d'ergonomie et ne devra jamais être considérée comme une protection suffisante à elle seule.

Les rôles métier identifiés dans le périmètre de l'application sont les suivants :

- **Admin**
- **OP-colis**
- **OP-stocks**

Les rôles étant cumulables, un même utilisateur pourra disposer de plusieurs habilitations en fonction de ses responsabilités au sein de l'entreprise.

Les habilitations principales associées à chaque rôle sont les suivantes :

Fonctionnalité	OP-stocks	OP-colis	Admin
Consulter le stock (lecture)	Oui	Non	Oui
Modifier une ligne de stock	Oui	Non	Oui
Déclencher la mise à jour annuelle	Oui	Non	Oui
Gérer le catalogue d'articles (CRUD)	Non	Non	Oui
Gérer les utilisateurs	Non	Non	Oui
Consulter les commandes	Non	Oui	Oui
Gérer les clients	Non	Oui	Oui
Exporter / imprimer les états	Oui	Oui	Oui

Chaque route API protégée devra effectuer les vérifications suivantes :

- présence d'un jeton d'authentification valide
- validité de sa signature
- validité temporelle du jeton, notamment sa date d'expiration
- présence du ou des rôles requis pour accéder à la ressource demandée
- autorisation d'accès à l'action métier concernée

Si le jeton est absent, invalide ou expiré, l'API retournera un code **401 Unauthorized**.

Si le jeton est valide mais que l'utilisateur ne dispose pas des droits suffisants pour accéder à la fonctionnalité demandée, l'API retournera un code **403 Forbidden**.

L'implémentation du contrôle d'accès sera réalisée côté back-end à l'aide de mécanismes adaptés à Flask, notamment par l'usage de décorateurs de protection des routes, de middlewares d'authentification et d'une vérification centralisée des rôles sur les endpoints métier.

Contraintes de sécurité complémentaires

Le protocole **HTTPS** sera obligatoire sur l'ensemble des environnements exposés, à l'exception éventuelle de l'environnement de développement local. Cette exigence vise à garantir le chiffrement des échanges entre le client et le serveur, notamment lors de la transmission des identifiants, des jetons d'authentification et des données métier.

En complément du mécanisme d'authentification et de gestion des autorisations, plusieurs mesures de sécurité devront être mises en place :

- **limitation du nombre de tentatives de connexion**, afin de réduire les risques d'attaques par force brute

- **journalisation des connexions et des échecs d'authentification**, afin de faciliter la détection d'activités anormales et l'analyse des incidents
- **invalidation des refresh tokens lors de la déconnexion**, afin d'empêcher leur réutilisation après fermeture de session
- **rotation des secrets de signature**, afin de limiter l'impact d'une éventuelle compromission
- **contrôle strict des droits d'accès sur l'ensemble des endpoints métier**, afin de garantir que chaque utilisateur n'accède qu'aux ressources correspondant à ses habilitations

Ces mesures ont pour objectif de renforcer la sécurité globale de l'application, de limiter les risques d'accès non autorisé et d'améliorer la traçabilité des événements sensibles.

Sécurité applicative

La sécurité applicative devra intégrer plusieurs mécanismes complémentaires afin de limiter les principaux risques liés à une application web exposant des données métier internes.

Risque	Mesure de protection
Injection SQL	Utilisation exclusive de l'ORM SQLAlchemy et de requêtes paramétrées. Aucune concaténation dynamique de chaînes ne devra être utilisée pour construire une requête SQL.
Cross-Site Scripting (XSS)	Échappement automatique des données dans le front-end Vue.js, validation des entrées côté back-end et mise en place d'une politique CSP (Content Security Policy) dans les en-têtes HTTP configurés via Gunicorn.
Cross-Site Request Forgery (CSRF)	L'utilisation d'un access token JWT transmis dans le header Authorization réduit fortement le risque de CSRF sur les appels API classiques. Si un refresh token est stocké dans un cookie, celui-ci devra être protégé par les attributs HttpOnly, Secure et SameSite, et les endpoints sensibles liés au renouvellement de session devront être sécurisés en conséquence.
Brute force sur la connexion	Limitation du nombre de tentatives de connexion, par exemple 5 tentatives maximum, suivie d'un blocage temporaire de 15 minutes ou d'un mécanisme équivalent.
Exposition de données sensibles	Les données personnelles ne devront pas apparaître dans les logs applicatifs. Les messages d'erreur retournés au client devront rester génériques et ne jamais exposer d'informations internes telles que des traces techniques, du SQL ou des détails d'implémentation.
Dépendances vulnérables	Audit régulier des dépendances front-end via npm audit et des dépendances back-end Python via un outil d'analyse de sécurité tel que pip-audit ou Safety, intégré au pipeline CI/CD.

Ces mesures viennent compléter la gestion de l'authentification et des autorisations, afin d'assurer un niveau de sécurité cohérent avec une application intranet manipulant des données internes et des données personnelles.

7. Plan de développement

Les choix techniques retenus pour le développement de l'application visent à répondre à quatre objectifs principaux : simplicité de mise en œuvre, maintenabilité, lisibilité de l'architecture et adéquation avec le périmètre fonctionnel. La stack sélectionnée repose sur des technologies modernes, largement documentées et adaptées à une application web intranet de gestion.

Composant	Technologie retenue	Alternatives écartées	Justification du choix
Framework front-end	Vue.js 3 avec Vite	React.js, Angular	Vue.js permet de développer une SPA claire, modulaire et facile à maintenir. Son intégration avec Vite offre une chaîne de développement rapide et légère, adaptée à un projet de taille modérée.
Gestion d'état	Pinia	Vuex, gestion locale uniquement	Pinia constitue la solution recommandée avec Vue 3 pour centraliser les états globaux de l'application, notamment la session utilisateur, les rôles et certaines données partagées.
Requêtes HTTP	Axios	Fetch API native	Axios facilite la centralisation des appels API, la gestion des erreurs, des timeouts et des intercepteurs, notamment pour l'injection automatique du jeton d'authentification dans les requêtes.
Style / UI	Tailwind CSS	Bootstrap, Bulma	Tailwind permet un contrôle précis de l'interface tout en limitant la quantité de CSS spécifique. Cette approche favorise un rendu moderne, cohérent et adaptable aux besoins métier.
Routage	Vue Router	Routage manuel	Vue Router constitue la solution standard pour organiser les vues, sécuriser certains accès côté client et structurer la navigation de l'application.
Bundler	Vite	Webpack, Parcel	Vite offre un démarrage rapide, un rechargement à chaud performant et une configuration légère, ce qui accélère le développement et simplifie la chaîne front-end.

Typage	TypeScript	JavaScript	TypeScript permet de réduire les erreurs de typage, d'améliorer la maintenabilité du code et de sécuriser les échanges entre composants et avec l'API.
Back-end	Python 3 + Flask	Django, FastAPI	Flask constitue un micro-framework léger, souple et adapté à une API REST modulaire. Il est cohérent avec le prototype existant et permet de construire un back-end métier clair sans surcomplexifier l'architecture.
ORM / accès aux données	SQLAlchemy	Requêtes SQL brutes, autres ORM	SQLAlchemy permet de structurer l'accès aux données, de sécuriser les requêtes et de conserver une bonne lisibilité du modèle applicatif.
Base de données	PostgreSQL	MySQL, MariaDB	PostgreSQL est une base relationnelle robuste, conforme aux standards SQL et particulièrement adaptée à une application métier nécessitant intégrité des données, requêtes complexes et évolutivité.

Ces choix permettent de construire une architecture homogène, moderne et réaliste, tout en limitant la complexité technique inutile. L'objectif n'est pas de multiplier les technologies, mais de s'appuyer sur une stack cohérente, facilement maintenable et adaptée au contexte fonctionnel de DigiCheese.

DoD / DoR

Definition of Ready (DoR)

Une user story est considérée comme prête à être intégrée dans un sprint lorsqu'elle satisfait l'ensemble des critères suivants :

- la user story est rédigée selon un format clair, de type : « *En tant que [rôle], je veux [action] afin de [bénéfice]* »
- les critères d'acceptation sont définis, complets et testables
- la user story a été estimée en points d'effort par l'équipe, notamment au travers d'un planning poker
- les maquettes, wireframes ou éléments d'interface nécessaires ont été validés par le Product Owner
- les endpoints API nécessaires sont identifiés et documentés, avec la méthode, l'URL et le format des données attendues
- les dépendances avec d'autres user stories sont connues et prises en compte dans la planification
- la user story est suffisamment découpée pour pouvoir être réalisée au cours d'un seul sprint

Definition of Done (DoD)

Une user story est considérée comme terminée lorsqu'elle satisfait l'ensemble des critères suivants :

- le code a été développé, relu et fusionné sur la branche **develop** via une Pull Request approuvée par au moins un autre membre de l'équipe
- le code respecte les conventions de qualité définies dans le projet, notamment en matière de formatage, de linting et de lisibilité
- les tests unitaires nécessaires ont été réalisés et validés ; la couverture du nouveau code atteint le seuil défini par l'équipe
- les critères d'acceptation ont tous été vérifiés et validés sur l'environnement d'intégration
- la fonctionnalité a été testée manuellement sur les navigateurs cibles du projet
- l'affichage et le comportement sont conformes aux exigences de responsive définies pour l'application
- aucune donnée sensible n'est exposée dans les logs, dans les réponses d'erreur ou dans les sorties techniques
- la documentation technique utile a été mise à jour
- la fonctionnalité a été validée par le Product Owner en environnement de recette

Organisation du développement

Le projet sera conduit selon la méthode **Agile Scrum**, afin de favoriser une livraison incrémentale, une bonne visibilité sur l'avancement et une capacité d'adaptation aux retours fonctionnels. Le développement sera organisé en **sprints de deux semaines**.

L'outil de gestion de projet retenu est **JIRA**, utilisé pour le suivi du backlog, la planification des sprints et la gestion des user stories. Le code source sera versionné dans un dépôt **GitHub**, utilisé pour la gestion des branches, des Pull Requests et des revues de code.

Les cérémonies Scrum retenues dans le cadre du projet sont les suivantes :

Cérémonie Scrum	Fréquence	Durée maximale	Participants
Sprint Planning	Début de chaque sprint	2 heures	Toute l'équipe + Product Owner
Daily Stand-up	Chaque jour ouvré	15 minutes	Toute l'équipe, Product Owner optionnel
Sprint Review	Fin de chaque sprint	1 heure	Toute l'équipe + Product Owner + client
Sprint Retrospective	Fin de chaque sprint	45 minutes	Toute l'équipe
Backlog Refinement	Milieu de sprint	1 heure	Toute l'équipe + Product Owner

Cette organisation a pour objectif d'assurer un pilotage régulier du projet, une priorisation continue du besoin et une amélioration progressive du fonctionnement de l'équipe.

8. Diagrammes et annexes

Diagrammes UML

Les diagrammes fournis en annexe décrivent le système selon plusieurs points de vue complémentaires : vue de contexte, vue fonctionnelle, vue métier, vue dynamique et vue des données. Ils servent de support de conception, de validation fonctionnelle et de communication entre les parties prenantes.

Diagramme de contexte général

Il présente l'environnement global de l'application DigiCheese et ses interactions avec les acteurs internes (Administrateur, Opérateur colis, Opérateur stock) et externes (Client, La Poste, Service mail, Base de données).

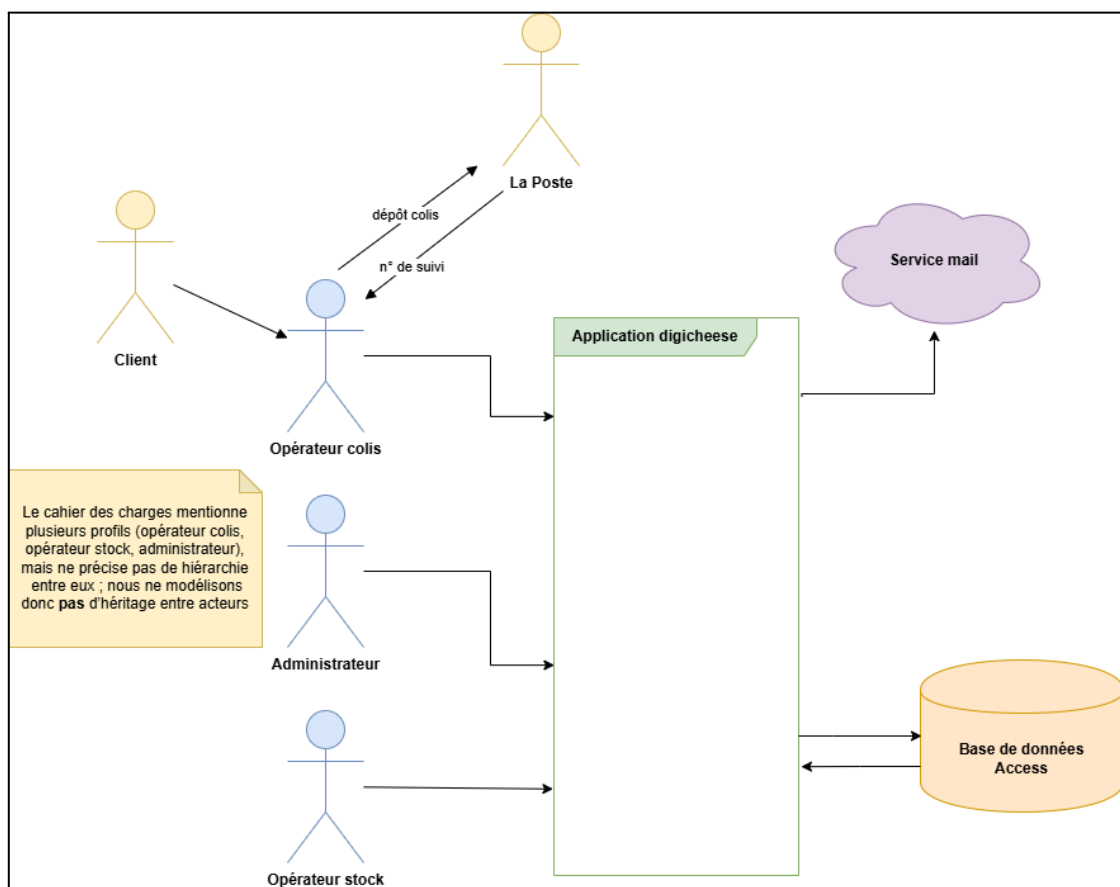


Diagramme de cas d'utilisation (Use Case)

Il décrit les principales fonctionnalités accessibles selon les rôles métier. Il met notamment en évidence les usages liés à la gestion des commandes, des clients, du conditionnement, du mailing, des impressions, des statistiques, des stocks et des inventaires.

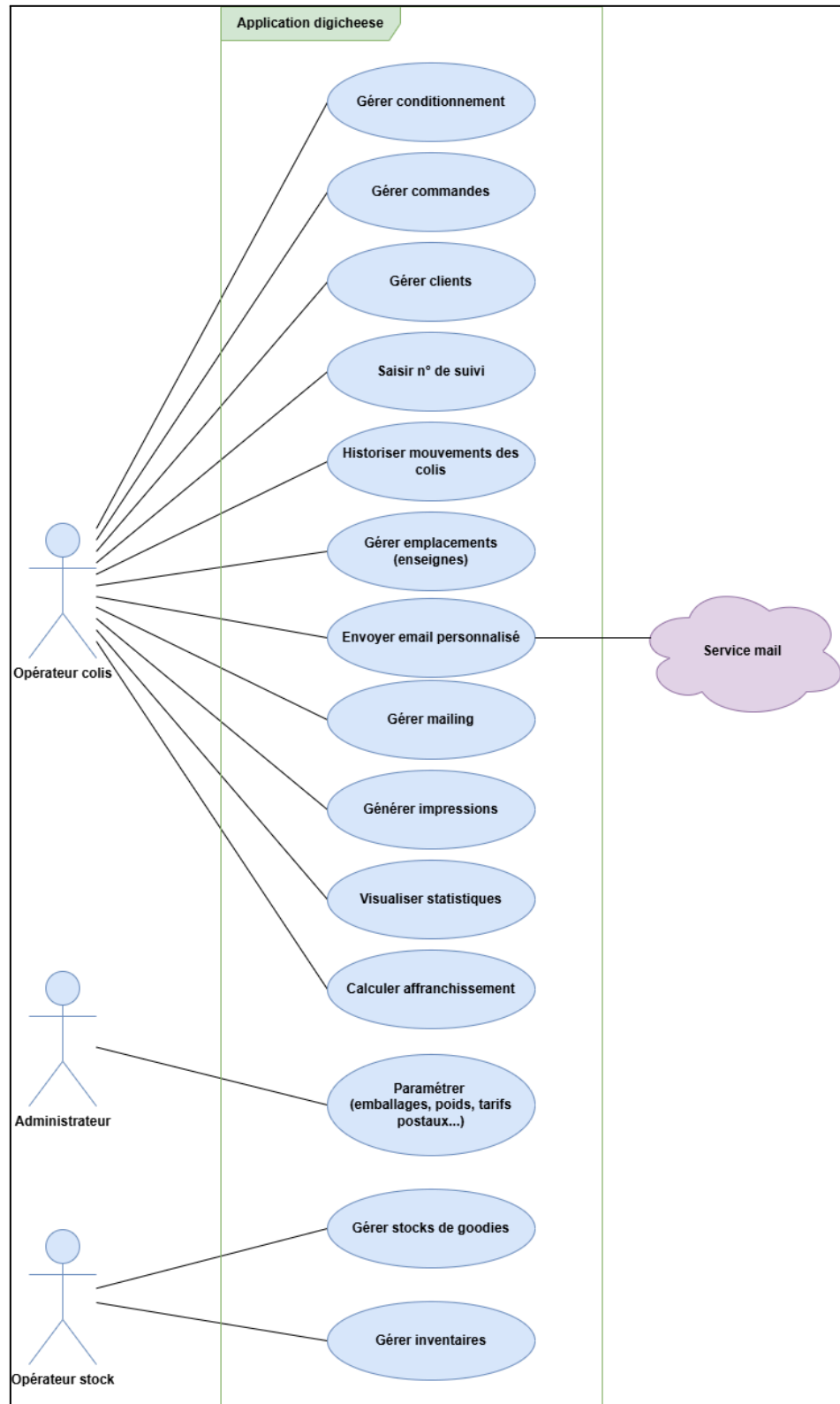


Diagramme de classes

Il représente la vue métier conceptuelle de l'application. Il modélise les principales classes du domaine, notamment Client, Commande, LigneCommande, Goodie, Conditionnement, TarifPostal, MouvementColis, Commune, Modele et Emplacement.

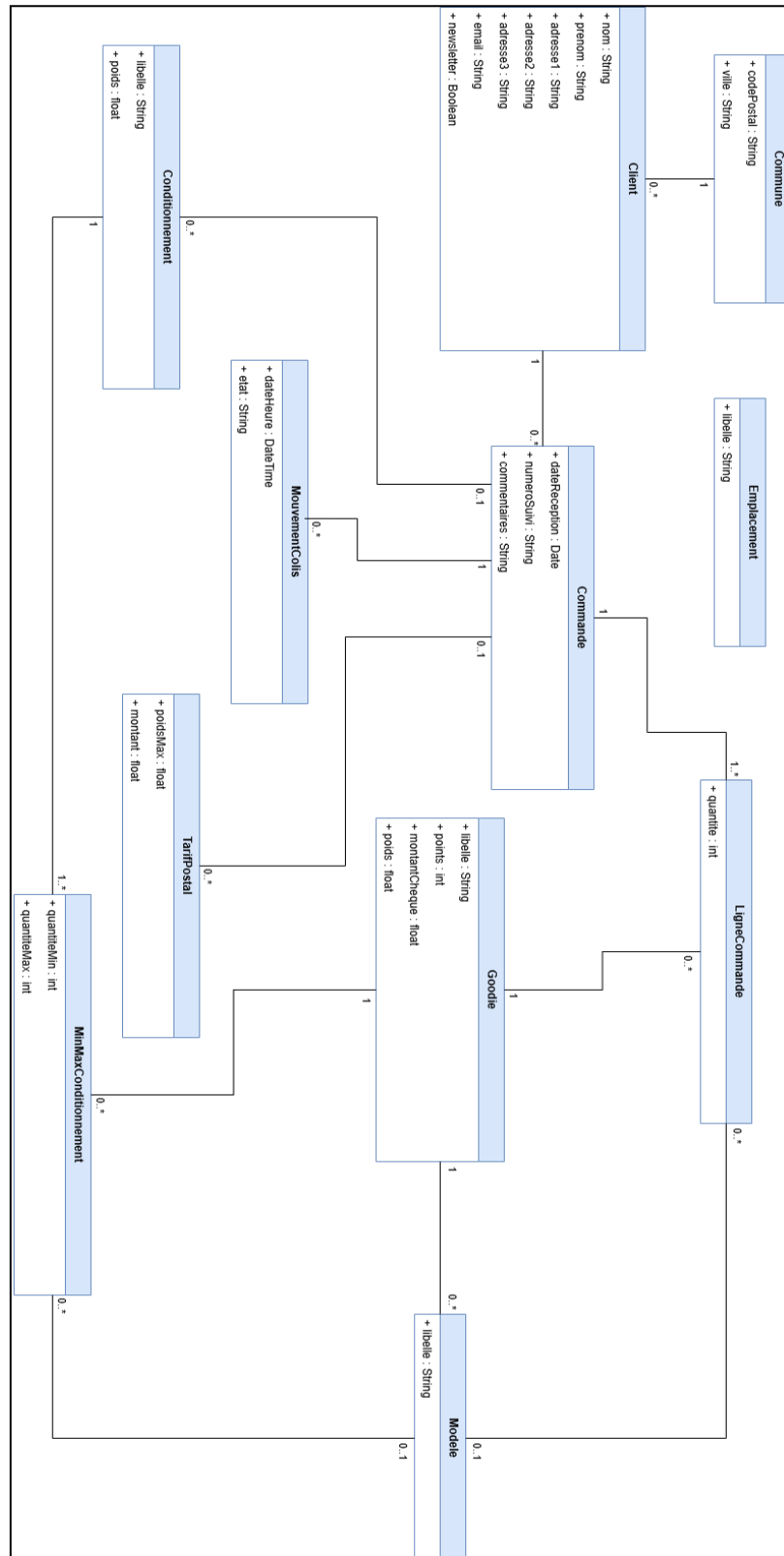


Diagramme de séquence

Il décrit le scénario de traitement d'une commande colis : recherche ou création du client, création de la commande, calcul du conditionnement, dépôt du colis, saisie du numéro de suivi, historisation du mouvement et envoi éventuel d'un email.

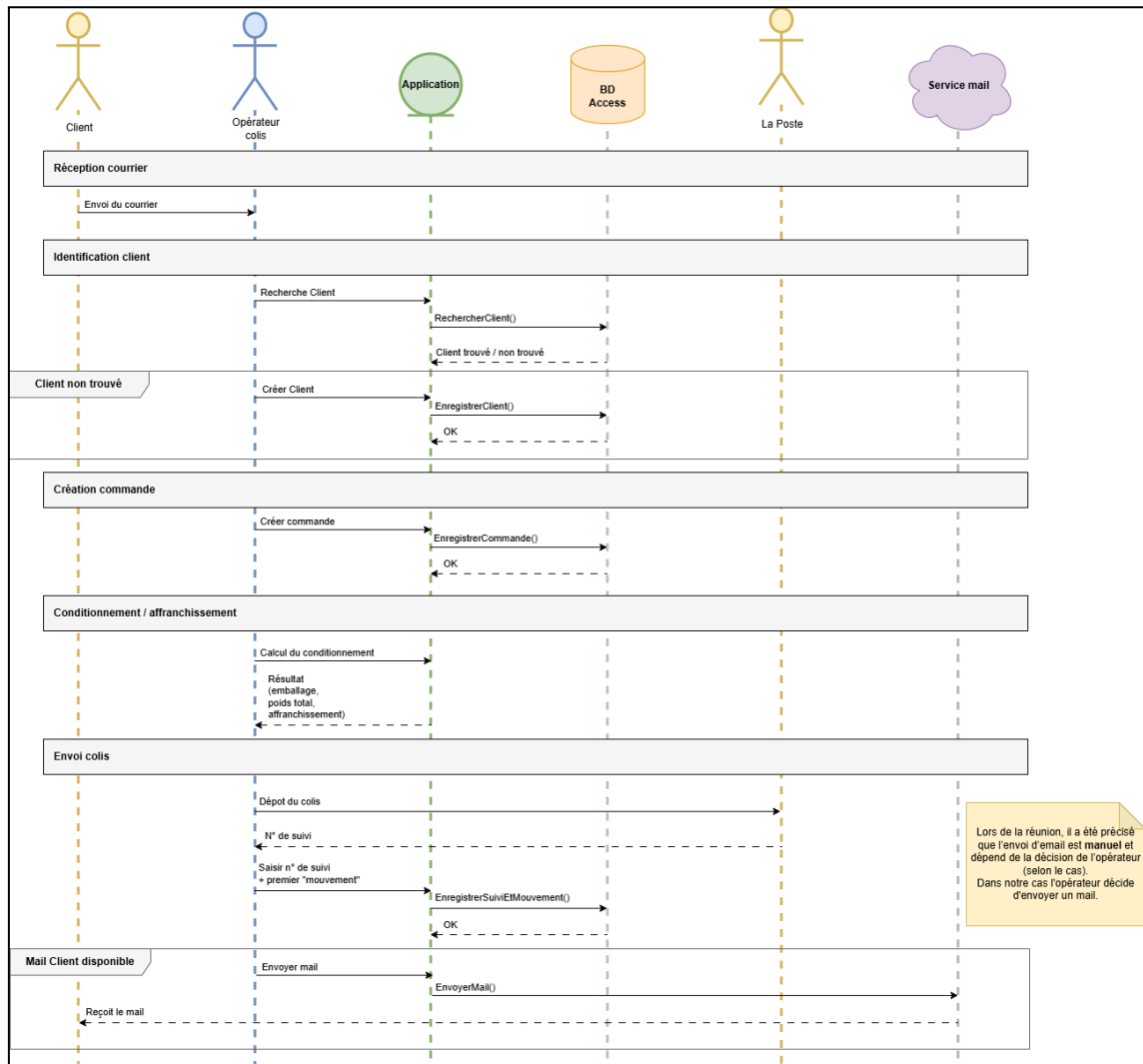


Diagramme entité-relation (ERD / MPD)

Ce diagramme représente le modèle physique des données de la base PostgreSQL. Il liste l'ensemble des tables, leurs colonnes (avec types SQL), clés primaires, clés étrangères et contraintes d'intégrité. Il est conforme au schéma produit lors du bloc de modélisation et est mis à jour pour intégrer le module de gestion des stocks.

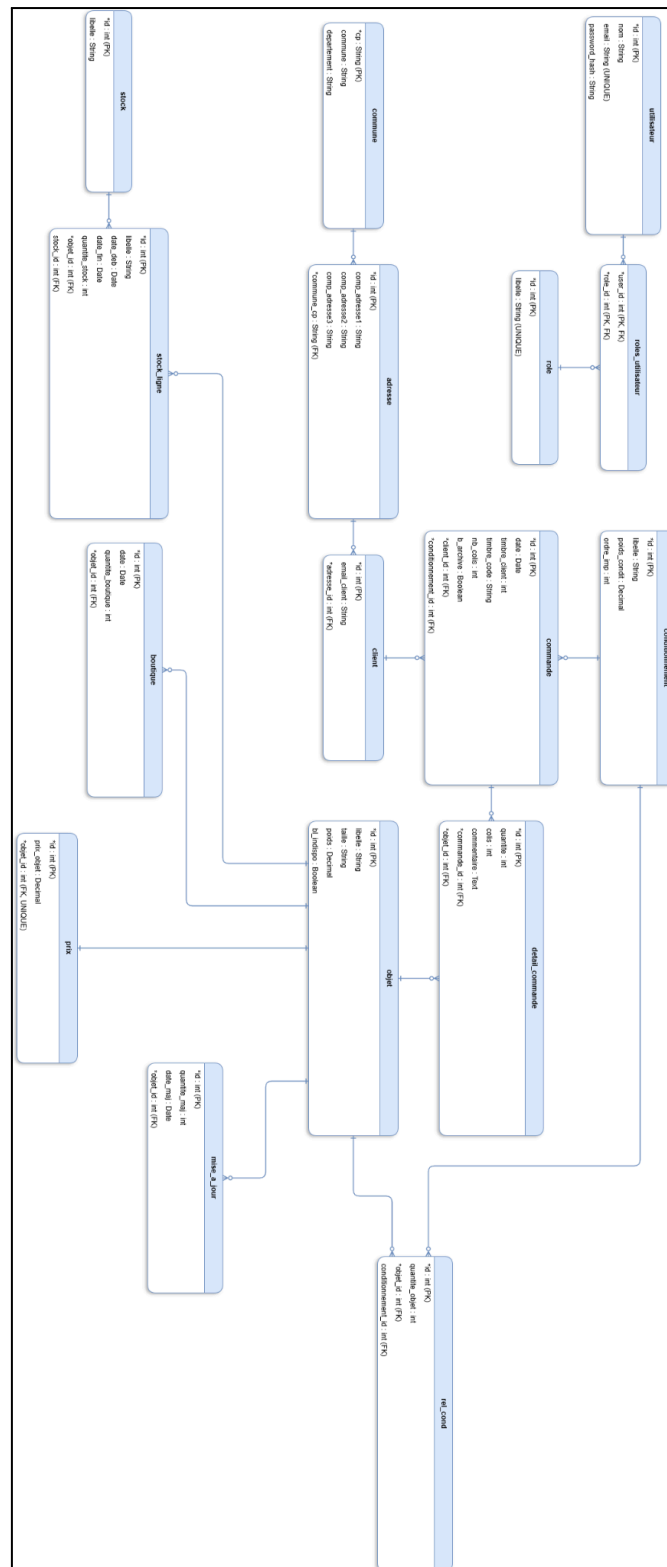


Diagramme BPMN

Le diagramme BPMN décrit le processus métier de traitement d'une commande par l'Opérateur colis. Il couvre les principales étapes du flux métier : réception du courrier, recherche ou création du client, création de la commande, saisie des goodies et quantités, calcul du conditionnement et de l'affranchissement, préparation du colis, dépôt à La Poste, récupération puis saisie du numéro de suivi, historisation du mouvement et envoi éventuel d'un email via le service mail.

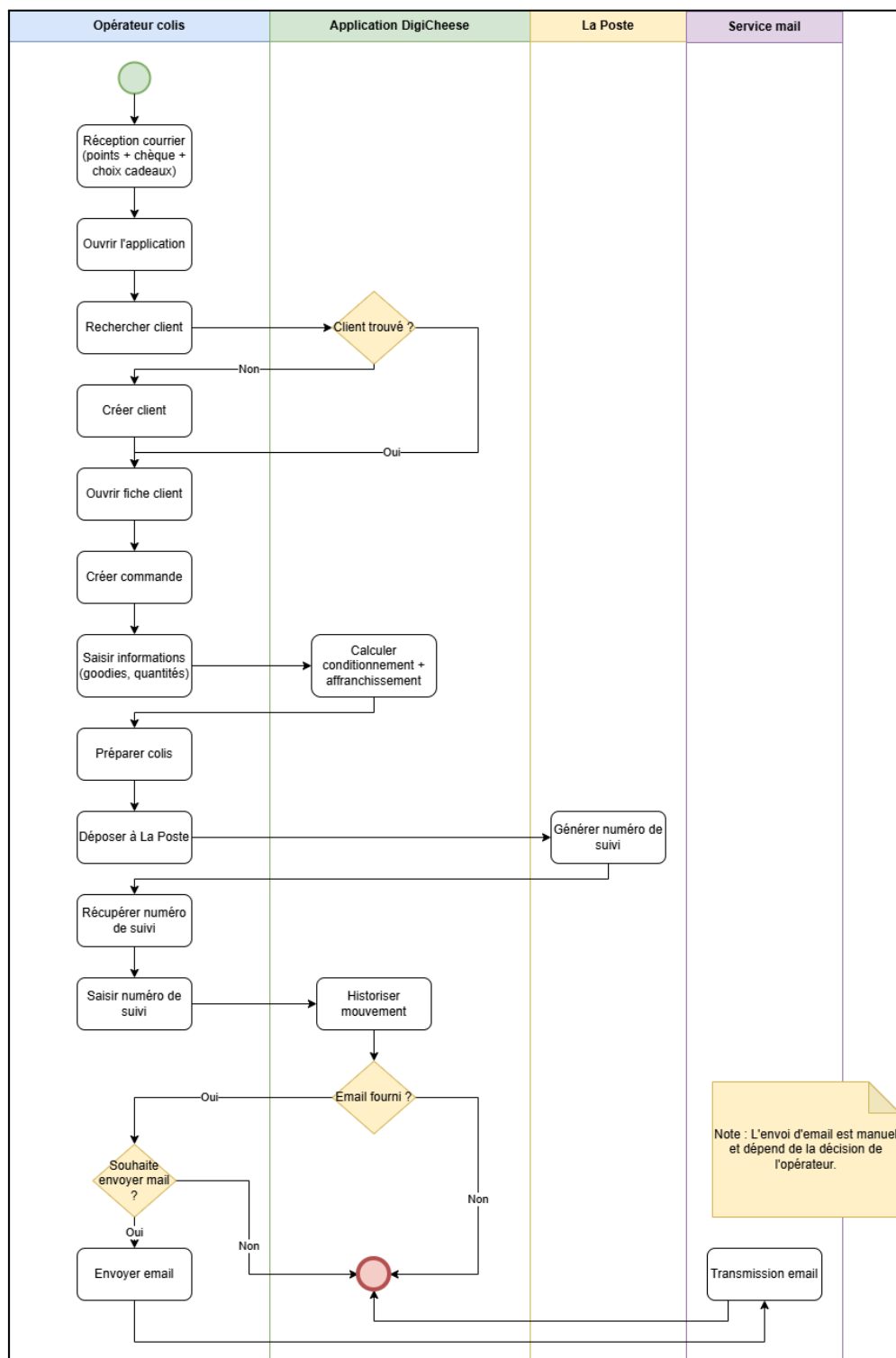
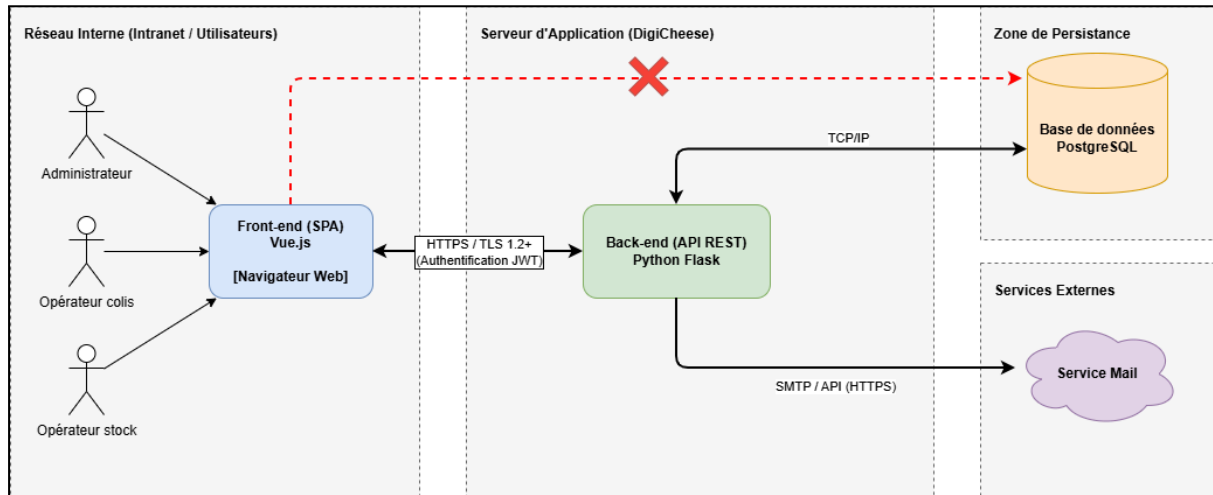


Schéma réseau

Le schéma réseau présente l'architecture technique cible de l'application DigiCheese selon une logique client-serveur 3-tiers. Les utilisateurs internes (Administrateur, Opérateur colis, Opérateur stock) accèdent à l'application via un navigateur web et interagissent avec le front-end SPA en Vue.js. Le front-end communique exclusivement en HTTPS / TLS 1.2+ avec le back-end Flask, qui centralise la logique métier, l'authentification JWT et les échanges avec les services techniques. Le back-end est le seul composant autorisé à accéder à la base de données PostgreSQL et au service mail externe. Le schéma souligne explicitement qu'aucun accès direct entre le navigateur et la base de données n'est autorisé.



Algorithmes

Les algorithmes ci-dessous décrivent les principaux traitements métier de l'application DigiCheese sous une forme textuelle simplifiée.

Algorithme 1 : Recherche ou création du client

```
Saisir les informations du client
Rechercher le client dans la base de données
Si client trouvé
    Ouvrir la fiche client
Fin
Sinon
    Créer le client avec les informations saisies
    Si erreur de création
        Afficher un message d'erreur
        Arrêter le traitement
    Fin
    Sinon
        Ouvrir la fiche du client nouvellement créé
    Fin
Fin
```

Algorithme 2 : Création de commande

```
Sélectionner le client
Créer une nouvelle commande
Si erreur de création
    Afficher un message d'erreur
    Arrêter le traitement
Fin
Sinon
    Saisir les goodies et les quantités
    Pour chaque goodie saisi
        Créer une ligne de commande
        Si erreur d'enregistrement
            Afficher un message d'erreur
            Arrêter le traitement
        Fin
    Fin
    Valider la commande
Fin
```

Algorithme 3 : Calcul du conditionnement et de l'affranchissement

```
Récupérer les lignes de commande
Calculer le poids total des goodies
Déterminer le conditionnement adapté
Ajouter le poids du conditionnement au poids total
Rechercher le tarif postal correspondant
Si aucun tarif trouvé
    Afficher un message d'erreur
    Arrêter le traitement
Fin
Sinon
    Retourner le conditionnement retenu
    Retourner le poids total calculé
    Retourner le tarif d'affranchissement
Fin
```

Algorithme 4 : Historisation du mouvement colis et envoi d'email

```
Saisir le numéro de suivi
Enregistrer le mouvement de colis
Si erreur d'enregistrement
    Afficher un message d'erreur
    Arrêter le traitement
Fin
Sinon
    Vérifier si une adresse email client existe
    Si email fourni
        Demander si l'opérateur souhaite envoyer un email
        Si oui
            Envoyer l'email
            Si erreur d'envoi
                Afficher un message d'erreur
            Fin
            Sinon
                Confirmer l'envoi
            Fin
        Fin
    Fin
Fin
```

Algorithme 5 : Mise à jour annuelle du stock

```
Sélectionner le stock de l'année précédente
Créer un nouveau stock annuel
Si erreur de création
    Afficher un message d'erreur
    Arrêter le traitement
Fin
Sinon
    Pour chaque ligne du stock précédent
        Créer une nouvelle ligne dans le stock annuel
        Reporter la quantité selon la règle définie
        Si erreur de création
            Afficher un message d'erreur
            Arrêter le traitement
        Fin
    Fin
    Valider le nouveau stock
Fin
```